Multithreading in C

Concepts, Lifecycle, Synchronization, Race Conditions, Mutex, Semaphores, and Thread-Safe Programming

OPERATING SYSTEMS

SYSTEM PROGRAMMING

tech: POSIX threads (pthread)

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

void* worker(void* arg) {
    printf("Thread active\\n");
    return NULL;
}

int main() {
    pthread_t tid;
    pthread_mutex_t lock;

    pthread_mutex_init(&lock, NULL);
    pthread_create(&tid, NULL, worker,
NULL);
    pthread_join(tid, NULL);
    return 0;
}
```

Learning Objectives

By the end of this lecture, students should be able to:

1. Define process, thread, concurrency, and multithreading clearly.
2. Explain the thread lifecycle in C with `pthread_create()` and `pthread_join()`.
3. Identify race conditions and critical sections in C programs.
4. Use mutexes, semaphores, and condition variables correctly in pthread programs.

What is a Thread?

1. A thread is the smallest unit of CPU execution inside a process.

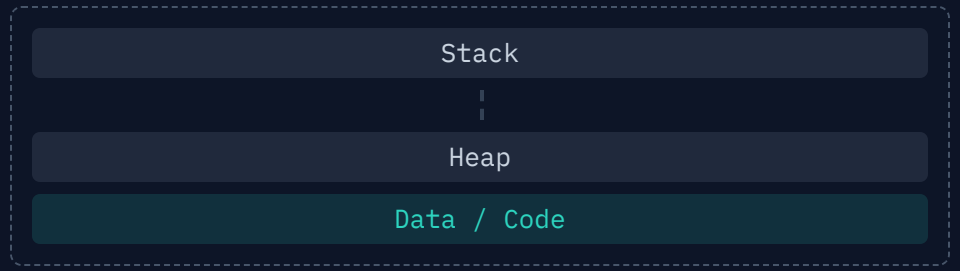
2. Threads in the same process share code, global data, and heap memory, but each thread has its own stack, program counter, and registers.

3. Multithreading means multiple threads execute concurrently within the same process.

Process vs Thread

Process

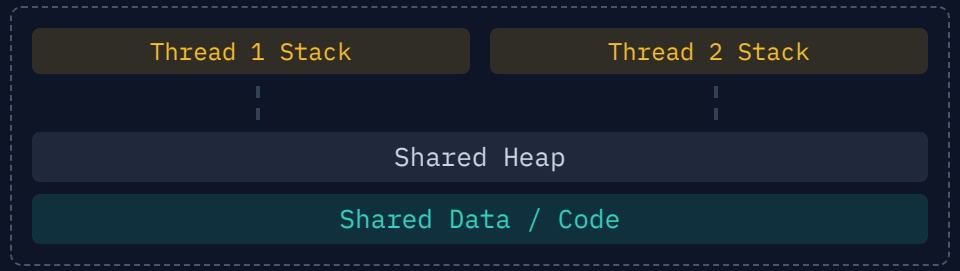
- Has its own address space and OS resources.
- Process creation is heavier (more overhead).
- Isolated memory; IPC (Inter-Process Communication) is complex.



VS

Thread

- Exists inside a process, shares most of that memory.
- Thread creation is lighter and faster in comparison.
- Communication is efficient, but synchronization becomes necessary.



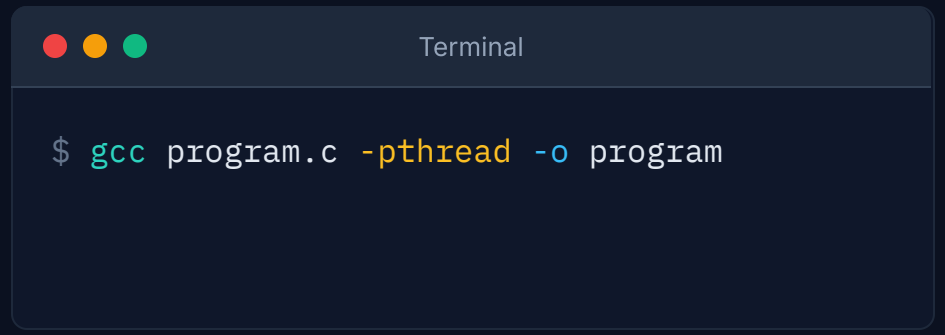
Why Use Multithreading in C?

1. To improve responsiveness, for example separating computation from I/O or UI-like tasks.
2. To improve throughput by allowing different tasks to progress concurrently.
3. To exploit multicore processors where multiple threads may run in parallel.

POSIX Threads Library

POSIX threads, commonly called **pthreads**, is a standard C API for thread creation and synchronization on Unix-like systems.

- Common functions include `pthread_create()`, `pthread_join()`, `pthread_mutex_lock()`, and `pthread_cond_wait()`.
- Programs using pthreads are typically compiled with the `-pthread` flag.



Creating a Thread in C

A new thread is created using `pthread_create()`. The thread starts executing a function whose signature is: `void* function(void*)`. `pthread_join()` makes one thread wait for another to finish.

thread_hello.c

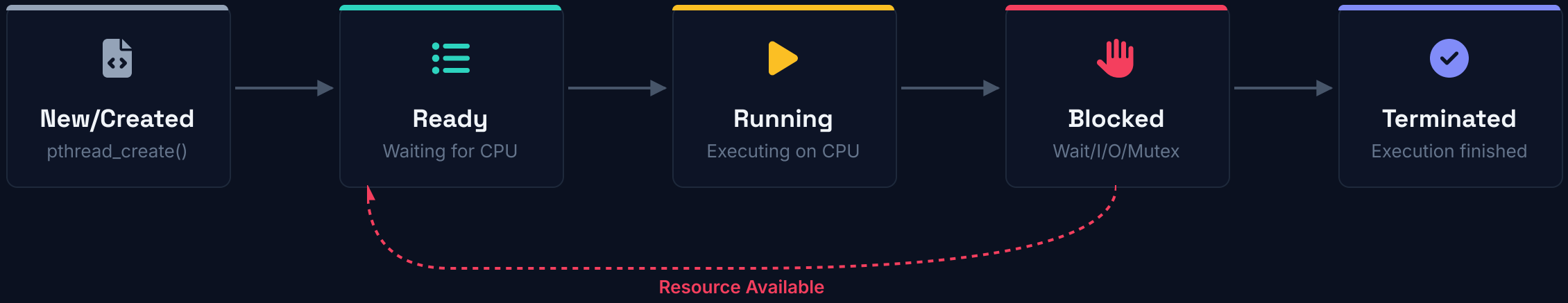
```c
#include <pthread.h>
#include <stdio.h>

void* worker(void* arg) {
    printf("Hello from worker thread\n");
    return NULL;
}

int main() {
    pthread_t tid;
    pthread_create(&tid, NULL, worker, NULL);
    pthread_join(tid, NULL);
    printf("Back in main thread\n");
    return 0;
}
```

Key Insights

- **`void*`** signature allows passing and returning generic data to the thread.
- **`pthread_join`** makes the main thread wait. Without it, the process might exit before the worker finishes.

Thread Lifecycle in C

- A thread typically moves through states such as **ready**, **running**, **blocked**, and **terminated**.
- After `pthread_create()`, the thread becomes ready/runnable; the scheduler eventually runs it.
- A thread may become blocked while waiting for I/O, a mutex, a semaphore, or another thread via `pthread_join()`.



Joining and Detaching Threads

- `pthread_join()` blocks the calling thread until the target thread terminates.
- Joinable threads can be waited on; detached threads release their resources automatically and cannot be joined later.
- Use join when the parent must collect completion or return status from a child thread.

join_example.c

```
pthread_t tid;
pthread_create(&tid, NULL, worker, NULL);
pthread_join(tid, NULL);   // wait for
worker to finish
```

Shared Data in Threads

- Global variables and heap data are shared among threads in the same process.
- If two threads update the same variable without coordination, the result may be wrong.
- This is the root of most multithreading bugs in C.

shared.c

```c
int counter = 0;
// shared by all threads
```

Critical Section

- A critical section is a part of code that accesses shared data and must not be executed by more than one thread at the same time.
- If a critical section is not protected, different threads can interfere with each other.
- Good synchronization ensures only one thread enters that code region when required.

critical.c

```
counter++; // unsafe if multiple threads
execute this together
```

Race Condition

A race condition occurs when multiple threads access shared data concurrently and the final result depends on timing or order of execution. They produce unpredictable behavior and are hard to reproduce.

race.c

```c
#include <pthread.h>
#include <stdio.h>

int counter = 0;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++; /* race condition */
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Counter = %d\n", counter);
    return 0;
}
```

Key Takeaways

- **Expected:** 200000
- **Actual:** usually less — `counter++` is NOT atomic.

What is a Mutex?

1. A mutex is a mutual-exclusion lock used to protect a critical section.
2. Only one thread can hold the mutex at a time; other threads must wait until it is unlocked.
3. Mutexes are mainly used to protect shared data from race conditions.

Using a Mutex in C

Lock before entering the critical section and unlock after leaving it. This makes the shared update effectively atomic.

mutex_counter.c

```c
#include <pthread.h>
#include <stdio.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* increment(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter = %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```

Key Insights

- **Result: always 200000**. The data race is completely prevented by the mutex.
- Always pair `lock` with `unlock`; destroy when done.

What is a Semaphore?

1. A semaphore is an integer-based synchronization primitive used to coordinate threads.

2. It is controlled through operations like `wait/decrement` and `signal/increment`.

3. Semaphores are commonly used for resource counting and signaling between threads, not just simple mutual exclusion.

Mutex vs Semaphore

Feature	Mutex	Semaphore
Purpose	Mutual exclusion	Signaling / resource counting
Value	Locked or unlocked	Integer count
Ownership	Yes	Usually no strict ownership
Common use	Protect shared data	Coordinate producers/consumers

Semaphore Example in C

A counting semaphore can limit how many threads use a resource simultaneously. Here, at most 2 threads enter the protected region at once.

semaphore.c

```c
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

sem_t sem;

void* task(void* arg) {
    int id = *(int*)arg;
    sem_wait(&sem);
    printf("Thread %d entered critical region\n", id);
    sleep(1);
    printf("Thread %d leaving critical region\n", id);
    sem_post(&sem);
    return NULL;
}

int main() {
    pthread_t t[4]; int id[4] = {1, 2, 3, 4};
    sem_init(&sem, 0, 2);
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, task, &id[i]);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    sem_destroy(&sem); return 0;
}
```

Key Insights

- **`sem_init(.., 2)`** → Grants 2 permits for threads to run simultaneously.
- **`sem_wait`** decreases available permits, **`sem_post`** increases permits after finishing.

Condition Variables

1. A condition variable lets threads sleep until a particular condition becomes true.

2. `pthread_cond_wait()` releases the mutex and blocks the thread; when awakened, it reacquires the mutex before returning.

3. Condition variables are useful in problems like producer-consumer where threads wait for *buffer-not-empty* or *buffer-not-full* conditions.

Producer–Consumer in C

Producer-consumer is a classic synchronization problem involving a shared bounded buffer.

- Producers add items; consumers remove them; synchronization prevents overflow, underflow, and races.
- Solved using a mutex plus condition variables, or semaphores plus a mutex.

producer.c

```
pthread_mutex_lock(&lock);
while (count == BUFFER_SIZE)
    pthread_cond_wait(&not_full, &lock);

buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;
count++;

pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&lock);
```

Thread-Safe Programming Best Practices

Define Thread-Safety

Ensure code behaves correctly when multiple threads execute it concurrently on shared data, avoiding unintended interactions.

Small Critical Sections

Keep protected regions as small as possible. Protect every shared mutable variable consistently and avoid ad hoc locking rules.

Clear System Designs

Minimize shared state and carefully document which lock protects which data. Prefer using one mutex per shared data structure.

Test Under Varied Schedules

Multithreaded bugs often appear only under specific execution orders. Repeatedly stress-test code under varied conditions to uncover race conditions.

Thank you for your attention